Lab 4

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

April 2026

Project Overview

This project explores Temporal-Difference (TD) learning methods, focusing on two fundamental control algorithms: SARSA and Q-learning. The objective is to understand how these algorithms learn optimal policies through interaction with an environment, specifically using the CliffWalking-v0 environment from Gymnasium.

The key goal is to analyze the difference between on-policy (SARSA) and off-policy (Q-learning) learning, and how this affects agent behavior, learning stability, and performance.

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/91f9c873f767025a209ca38b01085d079c12c75b/06_Reinforcement_Learning/Lab_4

Implementation Summary

Both SARSA and Q-learning were implemented using NumPy-based Q-tables indexed by state and action pairs. The agent interacts with the environment using an ε-greedy policy to balance exploration and exploitation.

The algorithms were implemented with online updates, meaning Q-values were updated after every step rather than at the end of each episode. This aligns with the core idea of Temporal-Difference learning, where updates are made incrementally using bootstrapping.

To ensure robust results, each algorithm was run across 30 different random seeds. Learning curves were generated by computing the average reward per episode along with 95% confidence intervals.

Key Results & Analysis

The learning curves show that both SARSA and Q-learning significantly improve performance over time, starting with very low rewards and converging toward near-optimal behavior.

Q-learning converges slightly faster and achieves marginally higher final rewards, indicating that it learns a more optimal policy. However, it also exhibits more variability during training.

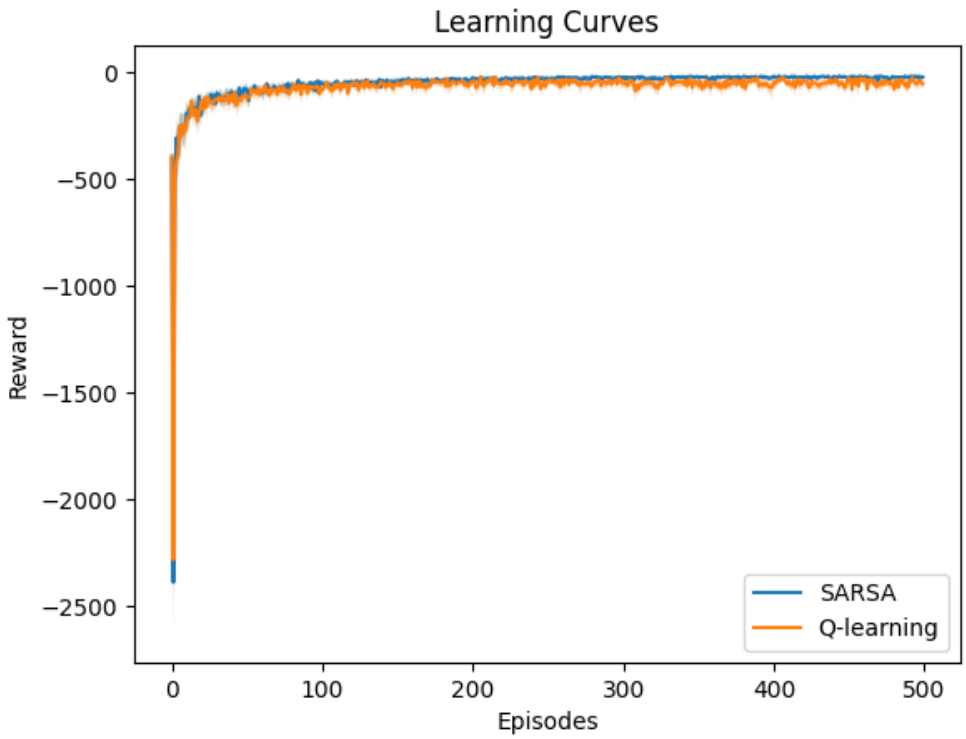
In contrast, SARSA demonstrates smoother and more stable learning behavior. While it may converge slightly slower, it avoids extreme negative rewards more effectively, especially during early exploration.

This difference is explained by their learning strategies. SARSA is an on-policy method that updates its Q-values based on the action actually taken, including exploratory actions. As a result, it learns safer policies that avoid risky states such as the cliff.

Q-learning, on the other hand, is an off-policy method that updates using the maximum possible future reward, regardless of the action taken. This leads to more aggressive optimization and a preference for the shortest path, even if it involves higher risk.

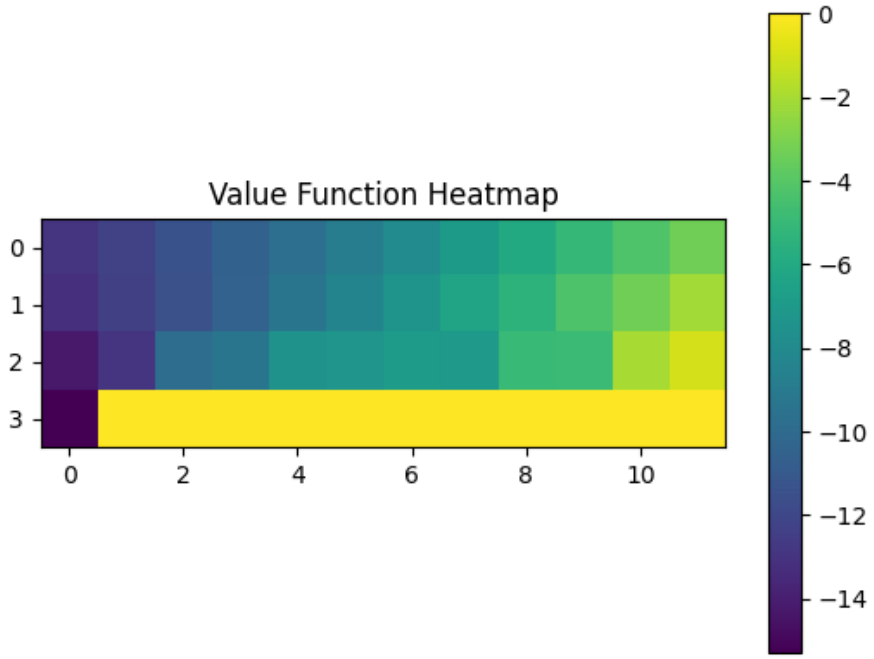
Visualizations

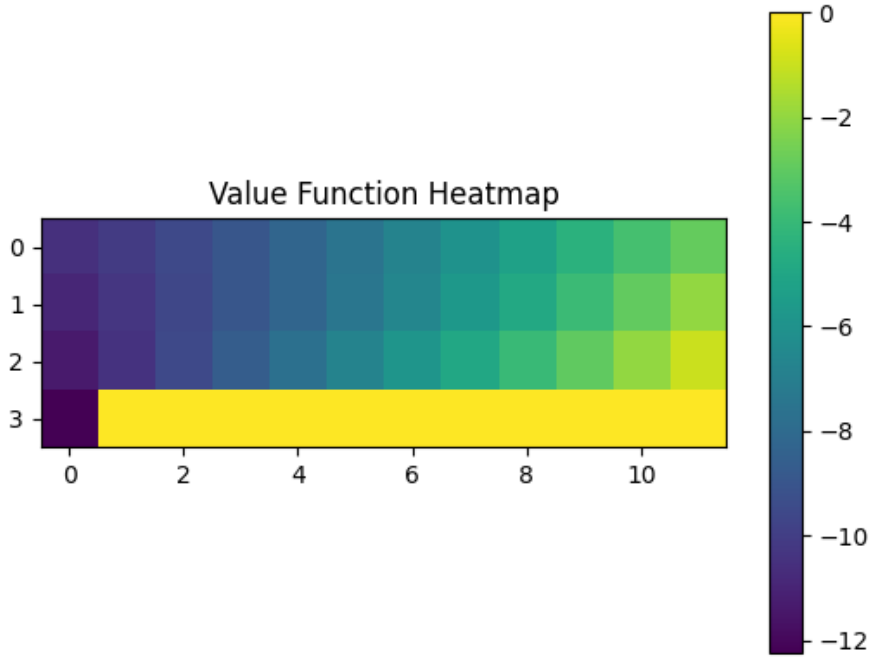
Learning Curves: Show the improvement in rewards over episodes, including confidence intervals:



- From the results, both SARSA and Q-learning improve significantly over time, moving from very low rewards at the beginning to near-optimal performance.
- Q-learning appears to converge slightly faster and reaches marginally higher rewards, indicating it learns a more optimal policy.
- SARSA, while slightly slower, shows smoother and more stable learning behavior, suggesting it is less sensitive to risky exploration.

Value Function Heatmaps: Illustrate how each algorithm values different states in the grid.



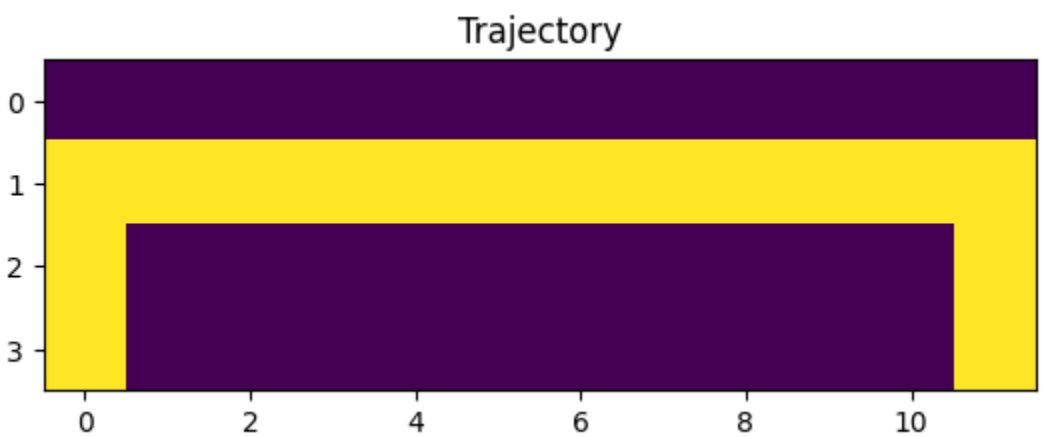


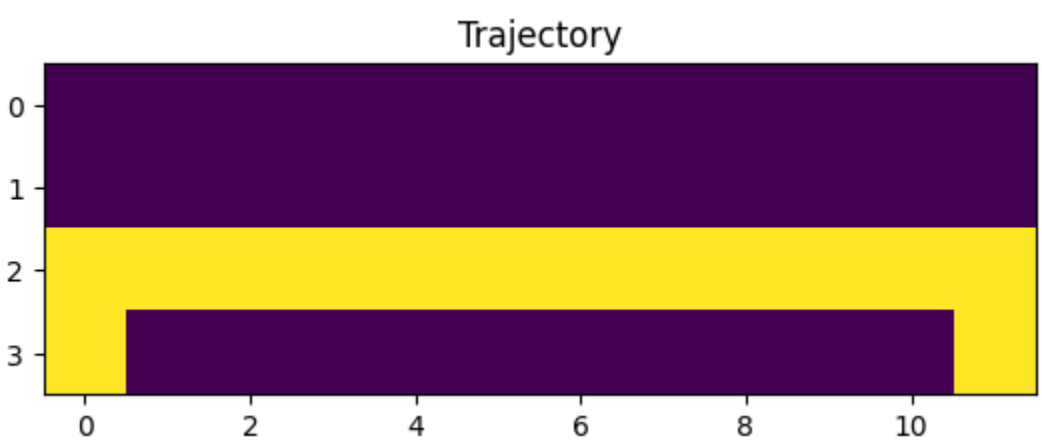
The policies learned suggest that both algorithms identify efficient paths to the goal. However, SARSA's policy tends to avoid the cliff region more, while Q-learning follows a more direct route, even if it is closer to danger.

Policy Representation: Displays the preferred action in each state.

The arrows demonstrate that SARSA learns a conservative and safer policy, while Q-learning learns a more optimal but riskier path.

Sample Trajectories: Demonstrate how the agent navigates from start to goal.





- The trajectories confirm the behavioral differences between the algorithms.
- SARSA follows a safer path that keeps distance from the cliff, reducing the risk of large negative rewards.
- Q-learning takes a shorter and more direct path toward the goal, which is optimal but passes closer to the cliff, making it riskier.

The visualizations confirm that SARSA tends to avoid the cliff by following a safer route, while Q-learning takes a more direct but riskier path closer to the cliff.

Conclusion

This project highlights the fundamental trade-off between safety and optimality in reinforcement learning. SARSA, as an on-policy method, produces safer and more stable behavior by incorporating exploration into its updates. In contrast, Q-learning prioritizes optimality, achieving higher rewards but at the cost of increased risk during learning.

These findings demonstrate the importance of selecting the appropriate learning strategy depending on the problem context, particularly in environments where safety is a critical concern.

AI Use Reflection

AI tools were used as a support resource throughout this assignment.

I asked:
"What is the difference between SARSA and Q-learning, and how should I implement them using Gymnasium?"

The AI explained that SARSA is an on-policy method, updating values based on the action actually taken, while Q-learning is off-policy, updating using the maximum possible next action. It also suggested structuring the implementation using standard Gymnasium loops with ε-greedy policies.

Initial Interaction:

During development, I used AI to refine both my understanding and implementation.

In one iteration, I had difficulty correctly implementing the Temporal-Difference update.

I asked:
"Am I applying the TD update correctly for SARSA and Q-learning?"

The AI suggested verifying that SARSA uses the next selected action a', while Q-learning uses the maximum over actions. After reviewing my code, I realized I was incorrectly using the max operator in both cases. Fixing this led to correct learning behavior and clearer differences between the algorithms.

In another iteration, I encountered confusion handling terminal states.

I asked:
"How should terminal states be handled in TD learning updates?"

The AI suggested setting the next state value to zero when the episode ends. After applying this, I avoided incorrect value propagation beyond terminal states

Use During Development:

AI was used to better understand how to implement Temporal-Difference updates, structure the training loops, and correctly handle elements such as ε-greedy policies, environment step outputs, and terminal states.

AI was also helpful in debugging minor issues and ensuring correct implementation of formulas for both algorithms.

Critical Evaluation:

While AI provided helpful guidance, its explanations were sometimes high-level and required careful interpretation.

For example, although it correctly described the difference between SARSA and Q-learning, it did not initially highlight how a small implementation mistake (using max in both) would eliminate the behavioral difference between the algorithms. I identified this issue by observing that both methods were producing nearly identical results, which contradicted theoretical expectations.

This demonstrated the importance of validating AI suggestions against expected algorithm behavior.

Learning Reflection

Through this process, I developed a clearer understanding of the fundamental difference between **on-policy (SARSA)** and **off-policy (Q-learning)** learning.

Debugging the update rules helped me understand how Q-learning tends to learn more optimistic policies by always assuming optimal future actions, while SARSA incorporates the effects of exploration through ε-greedy action selection.

This reinforced how small implementation details directly impact learning dynamics. I also learned that AI tools are most effective when used iteratively and critically, rather than relying on them for complete solutions.

Speaker Notes

- This project explores Temporal Difference learning using SARSA and Q-learning
- The goal is to compare on-policy vs off-policy learning
- We use the CliffWalking environment
- Both algorithms use Q-tables with NumPy
- We apply ε-greedy exploration
- We ran 30 seeds for reliable results
- Both algorithms improve over time
- Q-learning converges faster and reaches slightly higher rewards
- SARSA is more stable during training
- Heatmaps show value distribution across states
- Trajectories show how the agent moves
- SARSA avoids the cliff
- Q-learning takes a shorter but riskier path
- SARSA = on-policy - safer
- Q-learning = off-policy - more optimal
- Trade-off: safety vs performance
- SARSA is better for safety-critical environments
- Q-learning is better when optimal performance is the goal
- Choosing the right algorithm depends on the problem